National University of Lesotho

Department of Mathematics and Computer Science

CS4433 - Software Engineering -Individual Assignment

Time allowed: 7 weeks **Total marks: 100**

Date: 7th November 2026

Item	Details
Level	Final Project
Project type	Individual
Duration	7 weeks
Total marks	100
Development model	Waterfall, with V-Model verification and validation
Tools	UML tool, C++17 compiler, CMake, SQLite, Git with a hosted repository, a unit-testing framework (GoogleTest or Catch2), a static analyser (e.g. cppcheck or clang-tidy)

1. Aim and Learning Outcomes

You will take a realistic, moderately complex system through the full software engineering life cycle, making and justifying engineering decisions at each stage. On completion, you should be able to:

- Plan and manage a software project, including feasibility, scheduling and risk management.
- Elicit requirements from multiple stakeholders, resolve conflicts, and write a verifiable requirements specification.
- Produce a justified architecture and detailed design using UML, design patterns and a normalised database.
- Implement a maintainable, secure, object-oriented C++ application with persistent storage.
- Verify and validate software systematically using a documented test strategy and automated tests.
- Apply configuration management, continuous integration and quality assurance practices.
- Manage change after release and evaluate the professional, legal and ethical issues raised by the system.

2. Case Study: Moderated Learning Management System

A secondary school wants a Learning Management System (LMS) to manage courses, assignments and assessment. The school has four types of user: students, lecturers, Heads of Department (HoDs) and a system administrator. Every user logs in with a username and password and can only perform the actions allowed by their role.

- **Administrator:** manages user accounts, departments and courses, assigns lecturers to courses and enrols students.
- **Lecturer:** creates assignments with deadlines and a maximum mark, grants individual extensions, marks submissions and writes feedback.
- **Head of Department:** reviews and moderates the marks for all courses in their department and releases approved marks to students.
- **Student:** views enrolled courses and assignments, submits work, may resubmit before the deadline, and views released marks and feedback.

Each submission passes through a defined life cycle (for example: submitted, late, marked, moderated, released). All changes to marks must be recorded so that the school can see who changed a mark, when, and why.

Out of scope: online payment, video conferencing, automated grading, plagiarism detection, and web or mobile interfaces. The system will be a C++ application (console or simple desktop GUI) running on a single school computer or local network.

Stakeholder statements

The following statements were collected during initial interviews. They are incomplete, and some of them conflict. You must analyse, clarify and resolve them as part of requirements engineering.

> **Principal:** *"Teachers are not IT experts, so the system must be simple. Students must not see any mark until it has been checked by the Head of Department."*
>
> **Lecturer:** *"I want students to get their marks as soon as I have marked the work. Sometimes a student has a good reason for needing more time, and I should be able to give them an extension."*
>
> **Head of Department:** *"Late work should lose 10% of the available mark for each day it is late, for up to three days. After that it should receive zero. I need to see every mark in my department before it is released."*
>
> **Student representative:** *"Students should be able to fix a mistake and resubmit before the deadline, and we want written feedback, not just a number. Late work should not simply be rejected."*
>
> **IT officer:** *"Our computers are old and the internet is unreliable, so the system must work offline. Passwords must be protected, and the system should respond quickly even with all 800 students on it."*

3. Project Structure and Schedule

The project follows the Waterfall Model. Each phase ends with a documented deliverable and, at the three phase gates, a short review with your supervisor (formative, not separately marked). You may not begin implementation until your design has passed Gate 2.

Phase	Weeks	Main activities
Planning and management	1	Feasibility study, work breakdown, schedule, risk register
Requirements engineering	1–2	Stakeholder analysis, conflict resolution, SRS, use case model (Gate 1)
System design	2–3	Architecture, detailed UML design, design patterns, database design (Gate 2)
Implementation	4–5	C++ development with SQLite, CI pipeline, code reviews
Verification and validation	5–6	Unit, integration, system and acceptance testing (Gate 3)
Deployment	6	Packaging, installation guide, user manual, release v1.0
Maintenance	7	Change request, impact analysis, release v1.1, regression testing
Completion	7	Professional issues, final report, viva

4. Project Tasks

Phase 1: Project Planning and Management (10 marks)

- **Feasibility study (4 marks):** assess technical, operational, economic, legal and schedule feasibility and make a justified recommendation.
- **Work breakdown and schedule (3 marks):** produce a Work Breakdown Structure and a Gantt chart with dependencies, milestones and the critical path.
- **Risk management (3 marks):** produce a risk register of at least six risks (technical, project and people), with likelihood, impact, exposure, mitigation and owner. Update it at each phase gate.

Deliverable: *Project management plan.*

Phase 2: Requirements Engineering (14 marks)

- **Stakeholder analysis and conflict resolution (3 marks):** identify the stakeholders and their interests, list the conflicts and ambiguities in the stakeholder statements, and explain how you resolved each one.
- **Software Requirements Specification (5 marks):** write an SRS following a recognised structure (e.g. ISO/IEC/IEEE 29148). Include at least 12 functional and 6 non-functional requirements, each uniquely identified, prioritised (MoSCoW) and verifiable. Non-functional requirements must be measurable (e.g. "login completes within 2 seconds with 800 users stored", not "the system is fast").
- **Use case model (6 marks):** produce a Use Case Diagram covering all four actors, using include, extend and generalisation where appropriate. Write fully dressed use case specifications (preconditions, main flow, alternative and exception flows, postconditions, business rules) for three use cases, one of which must be "Submit assignment".

Deliverable: *Stakeholder analysis, SRS and use case model.*

Phase 3: System Design (24 marks)

- **Architecture (5 marks):** select and justify an architectural style (e.g. layered or Model–View–Controller). Provide a component diagram and a design rationale that discusses at least one alternative you rejected and why.
- **Class Diagram (7 marks):** produce a detailed design-level Class Diagram with visibility, types, multiplicities, inheritance and interfaces. It must apply at least two design patterns (e.g. Strategy for late-penalty rules, Repository/DAO for data access, Observer for notifications, Factory for user creation). Explain why each pattern was chosen and how your design follows SOLID principles.
- **Behavioural models (7 marks):** produce:
  - (i) a State Machine Diagram for the Submission life cycle;
  - (ii) an Activity Diagram with swimlanes for the end-to-end process from submission to released mark; and
  - (iii) Sequence Diagrams for "Submit assignment" (including late and resubmission paths) and "Moderate and release marks".
- **Database design (5 marks):** produce an ERD normalised to at least Third Normal Form, a data dictionary, and the SQL schema (tables, keys, constraints). Show how your tables map to your classes and justify any differences.

Deliverable: *Software design document.*

Phase 4: Implementation (21 marks)

Implement the system in C++17 according to your design. The implementation must:

- Implement all Must requirements from your SRS, including role-based access for all four user types.
- Apply the late-penalty, extension, resubmission and moderation rules you specified.
- Use an abstract User class with inheritance and polymorphism for the different roles, and implement the design patterns shown in your Class Diagram.
- Store data persistently in SQLite, accessed only through a data-access layer; use parameterised queries.
- Never store plain-text passwords: store salted hashes using a recognised library, and justify your choice.
- Record every mark change in an audit log (who, when, old value, new value, reason).
- Use exceptions and input validation so that invalid input or database errors do not crash the program.
- Build with CMake and follow a documented coding standard.

Marks are awarded for functionality (9), code quality and fidelity to the design (7), and persistence and security (5).

Deliverable: *Source code in a Git repository, with a build guide.*

Phase 5: Verification and Validation (12 marks)

- **Test strategy (3 marks):** write a test plan mapping each V-Model level (unit, integration, system, acceptance) to the artefact it verifies, with entry and exit criteria. Use equivalence partitioning and boundary value analysis, for example for deadlines and late penalties.
- **Automated unit and integration tests (5 marks):** write automated tests using GoogleTest or Catch2, run by your CI pipeline. Aim for at least 70% line coverage of the business-logic layer and report the actual figure.
- **System and acceptance testing (4 marks):** design and run system tests derived from your use case specifications, including exception flows. Keep a defect log showing severity, cause, fix and retest result.

Deliverable: *Test plan, test code, test results, coverage report and defect log.*

Phase 6: Configuration Management and Quality Assurance (6 marks)

- **Configuration management (3 marks):** use Git with a documented branching strategy (e.g. feature branches merged by pull request), meaningful commits and tagged releases. Set up a CI pipeline (e.g. GitHub Actions) that builds the project and runs the tests on every push.
- **Quality assurance (3 marks):** run a static analyser and address its findings; report code metrics (e.g. lines of code, cyclomatic complexity of key functions); and carry out a self code review against a checklist, recording what you changed as a result.

Deliverable: *Repository link, CI evidence, static analysis report, metrics and code review record.*

Phase 7: Deployment (4 marks)

Produce a UML Deployment Diagram, an installation guide that a school technician could follow, a user manual for each role, and release notes. Tag the release as v1.0.

Deliverable: *Deployment diagram, installation guide, user manuals and release notes.*

Phase 8: Maintenance (6 marks)

After release, the school submits the following change request:

> **CR1:** *"A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list."*

- **Classification and impact analysis (2 marks):** classify the maintenance type and justify it, then identify every affected requirement, diagram, class, table and test case.
- **Implementation (2 marks):** implement the change as release v1.1, updating the affected design documents.
- **Regression testing (2 marks):** add tests for the change and rerun the existing test suite to show that existing functionality still works.

Deliverable: *Change request analysis, v1.1 release and regression test results.*

Professional, Legal and Ethical Issues (3 marks)

Discuss the professional, legal and ethical issues raised by the system, including data protection of learners' personal information under applicable legislation, security of assessment records, accessibility for users with disabilities, and fairness of the late-penalty rules. Explain how your design addresses them.

5. Viva

Each student will attend a 15–20 minute viva in which they demonstrate the system, explain their design decisions and answer questions about their code. The viva is required. Where a student cannot satisfactorily explain their own work, marks for the related components may be withheld.

6. Consistency and Traceability

All artefacts must describe the same system. Provide a requirements traceability matrix linking each functional requirement to its use case, design elements (classes, diagrams, tables), source files and test cases. Marks will be reduced where code does not follow the design, where the database does not match the Class Diagram, where requirements are untested, or where artefacts do not correspond to the case study.

7. Submission Requirements

Submit a single report (PDF) and a link to your Git repository. The report should be 6,000–8,000 words, excluding diagrams, tables and appendices, and organised as follows:

No.	Report section
1	Title page, abstract and table of contents
2	Introduction
3	Project management plan
4	Requirements engineering (stakeholder analysis, SRS, use case model)
5	Software design document (architecture, UML, design patterns, database)
6	Implementation (structure, key design decisions, security, persistence)
7	Verification and validation
8	Configuration management and quality assurance
9	Deployment
10	Maintenance (change request CR1)
11	Professional, legal and ethical issues
12	Conclusion
13	References
14	Appendices (traceability matrix, full test results, user manuals, updated risk register)

Use a recognised referencing style consistently. Late submissions will be handled according to the institution's assessment policy.

8. Marking Scheme

Component	Marks
Phase 1: Project planning and management	10
Phase 2: Requirements engineering	14
Phase 3: System design (architecture 5, class 7, behavioural 7, database 5)	24
Phase 4: Implementation (functionality 9, quality 7, persistence/security 5)	21
Phase 5: Verification and validation	12
Phase 6: Configuration management and quality assurance	6
Phase 7: Deployment	4
Phase 8: Maintenance	6
Professional, legal and ethical issues	3
Total	**100**

9. Assessment Criteria

Each component will be judged against the following descriptors. At final year level, higher marks require justified decisions and critical insight, not only correct artefacts.

Level	Description
Outstanding (80–100%)	Complete, precise and professionally presented. Decisions are critically justified with alternatives considered. Artefacts are fully consistent and traceable. Shows insight beyond the brief.
Very good (70–79%)	Complete and accurate, with clear justification of most decisions and strong consistency across artefacts. Minor weaknesses only.
Good (60–69%)	Mostly complete and correct, with some justification. Some inconsistencies or gaps in depth.
Satisfactory (50–59%)	Meets the basic requirements but is largely descriptive, with limited justification, notational errors or noticeable inconsistencies.
Weak (40–49%)	Significant omissions or errors; weak links between artefacts; little justification.
Fail (below 40%)	Largely incomplete, incorrect or inconsistent with the case study.